Image, Videos and Filtering

This module assumes knowledge of the python programming language and familiarity with a python development environment. You should also be familiar with the concepts of OpenCV introduced in the previous practical.

We will be using OpenCV (version 4.1.x) computer vision library with Python 3.7.x. Whilst OpenCV is primarily aimed at computer vision tasks (i.e., image understanding and analysis), it is built on a central image processing and manipulation component that is both well-documented, open-source and free. It has already been setup for use on the Lab PCs (Windows and Linux) but you may alternatively use your own development environment on your own hardware. Instruction guides on installing OpenCV on your own hardware is provided on Ultra.

OpenCV has full documentation available at: https://docs.opencv.org/4.1.1/

Part 1: OpenCV I/O Functionality

In addition to loading images into our programs performing operations upon them and displaying them, it is also useful to be able to write result images back out to file. OpenCV is able to input and output a range of common image file formats.

OpenCV supports the following image formats:

- Windows bitmaps - BMP, DIB;
- JPEG files - JPEG, JPG, JPE / JPEG 2000 - JP2;
- Portable Network Graphics - PNG;
- Portable image format - PBM, PGM, PPM;
- Sun rasters - SR, RAS;
- TIFF files - TIFF, TIFF

The format for both the loading and saving of images is identified using a three- or four-letter file name extension. Run the **save_image.py** example from Ultra.

This program does not display the image in a window. Instead, it writes the result image out to a specified file location (the input and output files are specified in the source code). Ensure the input image is downloaded and present in the correct location before running the program.

As an example, this program simply inverts the image using the *bitwise_not()* function. Here, it outputs the result to a .png format file extension. Navigate to the folder of the Python script file (where the output image will be) and you can click on and view the image.

Try running the above program specifying ".tiff", ".png" and ".bmp" output file extensions. See what the size of the saved file is. Do this for each type of output file and compare the resulting file sizes. Which is most efficient?

Specific Tasks

1. With reference to the manual entry for *imwrite()*, experiment with changing both the cv2.IMWRITE_JPEG_QUALITY parameter when saving JPEG ".jpg" images and with cv2.IMWRITE_PNG_COMPRESSION when saving PNG ".png" images. What do you notice about the image quality and file sizes?

   Each is specified as a tuple forming the last parameter of *imwrite()* - e.g. :

   cv2.imwrite(...,..,(cv2.IMWRITE_JPEG_QUALITY, 75)).

2. Use the provided program (**save_image.py**) to invert a photographic image (e.g. "peppers.png" available on Ultra) and save the result out to file in JPEG format (call the result "inverted.jpg"). Now re-load this inverted image as a new file input to the same program to produce a secondary output image "inverted_back.jpg" (again use JPEG).

   Now investigate the use of the *absdiff()* function to write your own program that computes the difference between one image and another and displays the result. Try running this program on your original image ("peppers.png") and the image "inverted_back.jpg" that you created. You should see just a black image (as the inverted back image looks the same as the original) but are all the pixel values actually all zero?

   Use the earlier examples of accessing pixel elements to investigate this or perhaps multiply all the pixels values by a constant to make them brighter (The code in some of the lecture demos might be helpful here - https://github.com/atapour/ip-python-opencv).

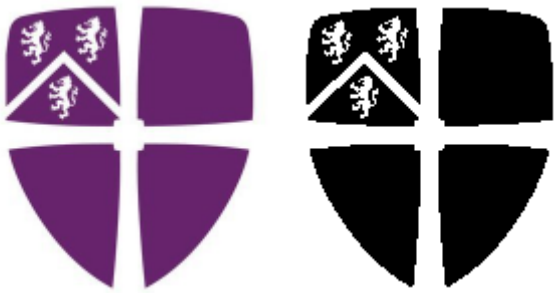
   Do the same but this time using PNG format images. What is the difference? Why does this occur for JPEG but not for PNG? (See if Google can give you an answer!)

   This demonstrates a key example of how some compression formats add noise into images.

Part 2: Logical Operations

By applying logical operations, write a program to imprint on the top left of an image the Durham University logo. At the end, your output should look something like this:

You can download the Durham University's logo (DULogo.png) from Ultra, together with a 'binary mask' of it (DULogoMask.png), i.e. an image with the same dimensions as the logo, where the logo's background pixels are white and the foreground pixels are black.



This mask was produced from the original logo by thresholding. That is, pixel values above a chosen threshold were put to 255 and any values below that threshold were put to 0.

Hint: Remember the "Region of Interest" concept from the last practical.

Part 3: Gamma Correction

Apply the gamma correction algorithm to the image available on Ultra ("peppers.png") as described in Lecture 3.

If you remember from the lecture, Gamma Correction assumes that all pixel values are real numbers in the range [0,1], so make sure this is the case before applying the transform. We looked at a demo of this (which is publicly available on GitHub - https://github.com/atapour/ip-python-opencv) but try to implement your own program without peeking first.

The outputs should look something like this:

$\gamma = 1/3$ $\gamma = 1$ (original) $\gamma = 3$

Part 4: Working with Videos

As well as being able to load images from file, it is also useful to be able to capture live images from an attached camera source. OpenCV provides a simple common interface to cameras attached to the computer. A single set of commands can be used to interface to different cameras regardless of difference in manufacturer, type, connection to PC and type of image returned. You have seen this in the demos during the lectures.

Run the program **capture_camera.py** example from Ultra. This should read the images from a video file (you can try the video provided on Ultra - **video.avi** (**`capture_camera.py video.avi`**) – or your own video) or an attached camera and display the video. The same script has been used in all the lecture demos, which you can find here: https://github.com/atapour/ip-python-opencv

You might want to "star" this repository (top of the GitHub page) for easier access for the duration of this module!

Specific Tasks

1. Experiment with the image processing functions we have looked at earlier to display live "processed" video. Try the following:
    - cv2.GaussianBlur() to produce a live blurred video
    - cv2.bitwise_not() to display a live inverted video

2. Change the "capture_camera.py" code so that extracts the 100×100 pixel area around the location of a mouse click and creates and displays this part of the original image in a new window. Remember how we learned about dealing with mouse clicks in OpenCV in the last practical.

3. Extend the "capture_camera.py" program to capture a sequence of frames (saved in numbered files) every:
    - 3 seconds
    - 10 seconds
    - 1 minute

    What would be a sensible choice of image file format given that we may now be collecting a large quantity of data?

4. Use your program to capture two successive frames of a static scene in your work area (i.e. without changes in camera position or any movement in the scene). You want to capture two image frames where you have changed the lighting conditions in the scene. You **don't** need to turn the lab lights off!!! - just cast a body shadow over the scene (staying out of shot) or shine a mobile phone screen/light into the shot.

    Use *cv2.absdiff()* to compare the original and inverted images to compute the difference between these two images. How are the two images different (or similar)?

Part 5: Creating Videos

To complement our ability to capture live video, it is also very useful (*and more efficient*!) to write video to file as a single video media file instead of a series of image frames. OpenCV gives us the facility to do this. Currently, OpenCV supports a number of video input/output formats, but provision does vary from operating system platform and sometimes dependant on the installation itself.

Generally, they are specified in the program code using four letter video codec (codec = "coder/decoder") identifiers such as the following:

```
cv2.VideoWriter_fourcc('X','V','I','D')    = XVID MPEG-4
cv2.VideoWriter_fourcc('P','I','M','1')    = MPEG-1 codec
cv2.VideoWriter_fourcc('M','J','P','G')    = motion-jpeg codec
cv2.VideoWriter_fourcc('M', 'P', '4', '2') = MPEG-4.2 codec
cv2.VideoWriter_fourcc('D', 'I', 'V', '3') = MPEG-4.3 codec
cv2.VideoWriter_fourcc('D', 'I', 'V', 'X') = MPEG-4 codec
cv2.VideoWriter_fourcc('U', '2', '6', '3') = H263 codec
cv2.VideoWriter_fourcc('I', '2', '6', '3') = H263I codec
cv2.VideoWriter_fourcc('F', 'L', 'V', '1') = FLV1 codec
 ....
```

See http://www.fourcc.org/codecs.php for all codecs...

These are many of the underlying video formats you use every day when watching videos on-line or similar. Which codes work on which operating systems platforms depends on the codecs you have installed on your system (which we are not getting into here) – "XVID" has been tested on the Durham systems.

Download the example **save_video.py** from Ultra. When you run this program, press the "x" key to terminate the program, which will end and save the recording.

This will create a (possibly large) video output file in the folder of your script file. By navigating to this directory, you should be able to play this file back in media player or vlc.

Experiment with selecting other video codecs as compressors from OpenCV but as they do not always work do not spend too long on this (often you may find they just output an empty file). Try experimenting with different resolutions and framerates as well.

Specific Task

1. Now that you can save videos, download the script "**gaussian_noise_filter.py**" from the lecture demos (https://github.com/atapour/ip-python-opencv) and alter the code to save the video to disk. Try the same with the "**salt_pepper_filter.py**" script from the same repository.